

melines.

- Individual release post entries merged for all relevant issues.
- By end of the day, milestone %x.y is expired.
- On or around Wednesday, the day before the release date:
 - Group Retrospective issues are updated with shipped and missed deliverables and team-members are tagged in the discussion.
- Wednesday, the day before the release date:
 - Milestone Cleanup runs on the schedule at Milestone cleanup schedule.
- Third Thursday of the release month: Release Day! ·
 - Release shipped to production.
 - Release post published.
- Friday immediately after release day:
 - The patch release process for the milestone starts. This includes regular and security patch releases.
 - All of the milestone's unfinished issues and merge requests are automatically moved to the next milestone, with the exception of ~security issues.
- On or around the Wednesday immediately following the release day:
 - Product plans are updated to reflect previous and current releases, including category epics and direction pages.
- On or around the second Monday following the release day:
 - Non-critical security patches are released.

Refer to release post content reviews for additional deadlines.

Note that deployments to GitLab.com are more frequent than monthly major/minor releases. See auto deploy transition guidance for details.

Kickoff

At the beginning of each release, we have a kickoff meeting, publicly livestreamed to YouTube. In the call, the Product Development team (PMs, Product Designers, and Engineers) communicate with the rest of the organization which issues are in scope for the upcoming release. The call is structured by product area with each PM leading their part of the call.

The Product Kickoff page is updated each month, which follows the content on the livestream.

Milestone Cleanup

Engineering Managers are responsible for capacity planning and scheduling for their respective teams with guidance from their counterpart Product Managers.

To ensure hygiene across Engineering, we run scheduled pipelines to move unfinished work (open issues and merge requests) with the expired milestone to the next milestone, and label ~"missed:x.y" for the expired milestone. Additionally, label ~"missed-deliverable" whenever ~"Deliverable" is presented.

This is currently implemented as part of our automated triage operations. Additionally, issues with the ~Deliverable label which have a milestone beyond current +1, will have the ~Deliverable label removed.

We keep the milestone open for 3 months after it's expired, based on the release and maintenance policy.

The milestone cleanup is currently applied to the following groups and projects:

- GitLab, schedule
- GitLab Runner, schedule
- GitLab Gitaly, schedule
- GitLab charts, schedule
- GitLab QA, schedule
- Omnibus GitLab, schedule (only moving milestone for now, not labelling)

Milestones closure is in the remit of the Delivery team. At any point in time a release might need to be created for an active milestone, and once that is no longer the case, the Delivery team closes the milestone.

Milestone cleanup schedule

The milestone cleanup will happen on the milestone due date.

These actions will be applied to open issues:

- Open issues and merge requests will be moved to the next milestone, and labeled with ~"missed:x.y".
- ~"missed-deliverable" will also be added whenever ~"Deliverable" is presented.

Milestones are closed when the Delivery team no longer needs to create a backport release for a specific milestone.

Use Group Labels and Group Milestones

When working in GitLab (and in particular, the GitLab.org group), use group labels and group milestones as much as you can. It is easier to plan issues and merge requests at the group level, and exposes ideas across projects more naturally. If you have a project label, you can promote it to a group milestone. This will merge all project labels with the same name into the one group label. The same is true for promoting group milestones.

Technical debt

We definitely don't want our technical debt to grow faster than our code base. To prevent this from happening we should consider not only the impact of the technical debt but also consider the impacts spreading like a contagion. How big and how fast is this problem going to be over time? Is it likely a bad piece of code will be copy-pasted for a future feature? In the end, the amount of resources available is always less than amount of technical debt to address.

To help with prioritization and decision-making process here, we recommend thinking about contagion as an interest rate of the technical debt. There is a great comment from the internet about it:

> You wouldn't pay off your \$50k student loan before first paying off your \$5k credit card and it's because of the high interest rate. The best debt to pay off first is one that has the highest loan payment to recurring payment reduction ratio, i.e. the one that reduces your overall debt payments the most, and that is usually the loan with the highest interest rate.

Technical debt is prioritized like other technical decisions in product groups by product management.

For technical debt which might span, or fall in gaps between groups they should be brought up for a globally optimized prioritization in retrospectives or directly with the appropriate member of the Product Leadership team. Additional avenues for addressing technical debt outside of product groups are Rapid Action issues and working groups.

Deferred UX

Sometimes there is an intentional decision to deviate from the agreed-upon MVC, which sacrifices the user experience. When this occurs, the Product Designer creates a follow-up issue and labels it Deferred UX to address the UX gap in subsequent releases.

For the same reasons as technical debt, we don't want Deferred UX to grow faster than our code base.

These issues are prioritized like other technical decisions in product groups by product management.

As with technical debt, Deferred UX should be brought up for globally optimized prioritization in retrospectives or directly with the appropriate member of the Product Leadership team.

UI polish

UI polish issues are visual improvements to the existing user interface, touching mainly aesthetic aspects of the UI that are guided by Pajamas foundations. UI polish issues generally capture improvements related to color, typography, iconography, and spacing. We apply the UI polish label to these issues. UI polish issues don't introduce functionality or behavior changes to a feature.

Examples of UI polish

- Aesthetic improvements (example): removing unnecessary borders from a UI, updating the background color of an element, fixing the font size of a heading element.
- Misalignment of text, buttons, etc (example): although because many times something isn't broken, these improvements are considered UI polish. These could also be considered a bug.
- Incorrect spacing between UI elements (example): when two interface elements are using inconsistent spacing values, such as 10px instead of 8px. It could also be considered technical debt. Note that if two interface elements have zero space between them, it's an obvious bug.
- Visual inconsistencies across different product areas (example): visual inconsistencies could occur when we have a series of buttons on a particular view. For example, when 3/4 of them have been migrated to use the Pajamas component, and 1/4 of them are still using a deprecated button, resulting in a visual inconsistency. This is considered a UI polish.

What is not UI polish

- Functional inconsistency related to the experience: for example, using a manual action to add an assignee automatically shows the assignee in the sidebar but using a manual action to add a weight to an issue does not automatically show the weight in the sidebar. This is not currently considered UI polish. It would be considered a UX issue.
- Improving visibility of system status: status indicator improvements are experience improvements and are not classified as UI polish.
 - Even when updating something that is purely visual, such as a status icon, to improve the meaning the user has of what they are viewing, we are trying to improve the experience of that user.

Monitor Merge Request Trends

Open merge requests sometimes become idle (not updated by a human in more than a month). Once a month, engineering managers will receive an Merge requests requiring attention triage issue that includes all (non-WIP/Draft) MRs for their group and use it to determine if any action should be taken (such as nudging the author/reviewer/maintainer). This assists in getting merge requests merged in a reasonable amount of time which we track with the Open MR Review Time (OMRT) and Open MR Age (OMA) performance indicators.

Open merge requests may also have other properties that indicate that the engineering manager should research them and potentially take action to improve efficiency. One key property is the number of threads, which, when high, may indicate a need to update the plan for the MR or that a synchronous discussion should be considered. Another property is the number of pipelines, which, when high, may indicate a need to revisit the plan for the MR. These metrics are not yet included in an automatically created a triage issue.

Security is everyone's responsibility

Security is our top priority. Our Security Team is raising the bar on security every day to protect users' data and make GitLab a safe place for everyone to contribute. There are many lines of code, and Security Teams need to scale. That means shifting security left in the Software Development LifeCycle (SDLC). Each team has an Application Security Stable Counterpart who can help you, and you can find more secure development help in the sec-appsec Slack channel.

Being able to start the security review process earlier in the software development lifecycle means we will catch vulnerabilities earlier, and mitigate identified vulnerabilities before the code is merged. You should know when and how to proactively seek an Application Security Review. You should also be familiar with our Secure Coding Guidelines.

We are fixing the obvious security issues before every merge, and therefore, scaling the security review process. Our workflow includes a check and validation by the reviewers of every merge request, thereby enabling developers to act on identified vulnerabilities before merging. As part of that process, developers are also encouraged to reach out to the Security Team to discuss the issue at that stage, rather than later on, when mitigating vulnerabilities becomes more expensive. After all, security is everyone's job. See also our Security Paradigm.

Rapid Engineering Response

From time to time, there are occasions that engineering team must act quickly in response to urgent issues. This section describes how the engineering team handles certain kinds of such issues.

Scope

Not everything is urgent. See below for a non-exclusive list of things that are in-scope and not in-scope. As always, use your experience and judgment, and communicate with others.

- In Scope

- Last-minute release blocking bug or security patch before an imminent release.
- High severity (severity::1/priority::1) security issues. Refer to security severity and priority.
- Highest priority and severity customer issues based on the priority and severity definitions.

- Not In Scope

- An operational issue of GitLab.com or a self managed customer environment. This falls under the on-call process.
- Self developed and maintained tools that are not officially supported products by GitLab.
- Feature request by a specific customer.

Process

1. Person requesting Rapid Engineering Response creates an issue supplying all known information and applies priority and severity (or security severity and priority) to the best of their ability.

1. Person requesting Rapid Engineering Response raises the issue to their own manager and the subject matter domain engineering manager (or the delegation if OOO).

1. In case a specific group cannot be determined, raise the issue to the Director of Engineering (or the delegation if OOO) of the section.

1. In case a specific section cannot be determined, raise the issue to the Sr. Director of Development (or the delegation if OOO).

1. The engineering sponsor (subject matter Manager, Director, and/or Sr. Director) invokes all stakeholders of the subject matter as a rapid response task force to determine the best route of resolution:

1. Engineering manager(s)

1. Product Management

1. QE

1. UX

1. Docs

1. Security

1. Support

1. Distribution engineering manager

1. Delivery engineering manager (Release Management)

1. Adjust priority and severity or security severity and priority if necessary, and work collaboratively on the determined resolution.

Performance Refinement

A bi-weekly performance refinement session is held by the Development and QE teams jointly to

raise awareness and foster wider collaboration about high-impact performance issues. A high impact issue has a direct measurable impact on GitLab.com service levels or error budgets.

Scope

The Performance Refinement issue board is reviewed in this refinement exercise.

Process

1. To participate in the bi-weekly refinement, ask your engineering director to forward the invite of the Performance Refinement meeting which is at 15:00 UTC every other Thursday. Here is the meeting agenda.
1. To nominate issues to the board:
 1. Assign a performance severity on the issue to help assess the priority assignment for the refinement session.
 1. Ensure that the issue clearly explains the problem, the (potential) impact on GitLab.com's availability, and ideally, clearly defines a proposed solution to the problem.
 1. Use the label bug::performance.
1. For the issues under the Open column:
 1. An engineering manager will be assigned if either the Milestone or the label workflow::ready for development is missing.
 1. Engineering manager brings assigned issue(s) to the Product Manager for prioritization and planning.
 1. Engineering manager unassigns themselves once the issue is planned for an iteration, i.e. associated with a Milestone and the label workflow::ready for development.
1. To highlight high-impact issues which need additional discussion, please add an agenda item.
1. If a guest attendee would be helpful for collaboration, please forward the invite. For example, a CSM or Support Engineer may have information that would be helpful for an upcoming topic.

Infradev

The infradev process is established to identify issues requiring priority attention in support of SaaS availability and reliability. These escalations are intended to primarily be asynchronous as timely triage and attention is required. In addition to primary management through the Issues, any gaps, concerns, or critical triage is handled in the SaaS Availability weekly standup.

Scope

The infradev issue board is the primary focus of this process.

Roles and Responsibilities

Infrastructure

1. Nominate issues by adding Infradev label.
1. Assess Severity and Priority and apply the corresponding label as appropriate.
1. Provide as much information as possible to assist development engineering troubleshooting.

Development

1. Development directors are responsible for triaging Infradev issues regularly by following the triage process below.

1. Development managers are encouraged to triage issues regularly as well.

1. Development managers collaborate with their counterpart Product Managers to refine, schedule, and resolve Infradev issues.

1. Usually, issues are nominated as Infradev issues by SREs or Managers in the Infrastructure Department. Development engineers/managers are not expected to nominate Infradev issues.

1. However, when it's necessary to spin off new issues from an existing Infradev issue, development engineers and managers may also apply Infradev label to the new issues.

1. When development engineers and managers split off new Infradev issues, they must have a Severity and Priority labels to the new issues. The labels should correspond to the importance of the follow-on work.

Product Management

1. Product Managers perform holistic prioritization of both product roadmap and Infradev issues as one unified backlog.

1. Product Managers collaborate with their counterpart Development Managers to refine, schedule, and resolve Infradev issues.

Triage Process

(To be completed primarily by Development Engineering Management)

Issues are nominated to the board through the inclusion of the label infradev and will appear on the infradev board.

1. Review issues in the Open column. Look for issues within your Stage/Group/Category, but also for those which lack a clear assignment or where the assignment may need correction.

1. Review the severity on the issue to validate appropriate prioritization.

1. Ensure that the issue clearly explains the problem, the (potential) impact on GitLab.com's availability, and ideally, clearly defines a proposed solution to the problem.

1. Assign a Development Manager and a Product Manager to any issue where the Milestone or the label workflow::ready for development is missing.

1. Development Manager and Product Manager collaborate on the assigned issue(s) for prioritization and planning.

1. Development Manager and Product Manager unassign themselves once the issue is planned for an iteration, i.e. associated with a Milestone and the label workflow::ready for development.

1. All Issues should be prioritized into the appropriate workflow stage. It is the intent to maintain no Open (un-triaged) items.

Issues with ~infradev ~severity::1 ~priority::1 ~production request labels applied require immediate resolution.

~infradev issues requiring a ~"breaking change" should not exist. If a current ~infradev issue requires a breaking change then it should split into two issues. The first issue should be the immediate ~infradev work that can be done under current SLOs. The second issue should be ~"breaking change" work that needs to be completed at the next major release in accordance with

deprecation guidance. Agreement from development DRI as well as the infrastructure DRI should be documented on the issue.

Infradev issues are also shown in the monthly Error Budget Report.

A Guide to Creating Effective Infradev Issues

Triage of infradev Issues is desired to occur asynchronously. These points below will ensure that your infradev issues gain maximum traction.

1. Use the InfraDev issue template to create the issue on the [gitlab-org/gitlab](#) issue tracker.

1. Clearly state the scope of the problem, and how it affects GitLab.com. Examples could include:

1. Reliability issues: the problem could cause a widespread outage or degradation on GitLab.com. example

1. Saturation issues: the problem could lead to increased saturation, latency issues due to resource over-utilization. example

1. Service-level degradation: the problem is causing our service-level monitoring to degrade, impacting the overall SLA of GitLab.com and potentially leading to SLA violations. example

1. Unnecessary alerts: the problem does not have a major impact on users, but is leading to extraneous alerts, impacting the ability of SREs to effectively triage incidents due to alerting noise. example

1. Problems which extend the time to diagnosis of incidents: for example, issues which degrade the observability of GitLab.com, swallow user-impacting errors or logs, etc. These could lead to incidents taking much longer to clear, and impacting availability. example

1. Deficiencies in our public APIs which lead to customers compensating by generating substantially more traffic to get the required results. example

1. Quantify the effect of the problem to help ensure that correct prioritization occurs.

1. Include costs to availability. The Incident Budget Explorer dashboard can help here.

1. Include the number of times alerts have fired owing to the problem, how much time was spent dealing with the problem, and how many people were involved.

1. Include screenshots of visualization from Grafana or Kibana.

1. Always include a permalink to the source of the screenshot so that others can investigate further.

1. Provide a clear, unambiguous, self-contained solution to the problem. Do not add the infradev label to architectural problems, vague solutions, or requests to investigate an unknown root-cause.

1. Ensure scope is limited. Each issue should be able to be owned by a single stage group team and should not need to be broken down further. Single task solutions are best.

1. Ensure a realistic severity is applied: review the availability severity label guidelines and ensure that applied severity matches. Always ensure all issues have a severity, even if you are unsure.

1. If possible, include ownership labels for more effective triage. The product categories can help determine the appropriate stage group to assign the issue to.

1. Cross-reference links to Production Incidents, PagerDuty Alerts, Slack Alerts and Slack Discussions. To help ensure that the team performing the triage have all the available data.

1. By adding "Related" links on the infradev issue, the Infradev Status Report will display a count of the number of production incidents related to each infradev issue, for easier and clearer prioritization.

1. Ensure that the issue title is accurate, brief and clear. Change the title over time if you need to keep it accurate.
1. By adding an infradev label to an issue, you are assuming responsibility and becoming the sponsor/champion of the issue.
1. Provide a method for validating that the original issue still exists
 1. Sometimes infradev issues will resolve on their own, or are resolved as a side-effect of an unrelated change.
 1. In the infradev issue description, provide a clear way of checking whether the problem still exists.
 1. Having a way of checking validity can save on a great deal of back-and-forth discussion between Infradev Triage participants including Engineering Managers, Directors and Product Managers and make space for other non-resolved issues to get scheduled sooner.
 1. Ideally, provide a link to a Grafana query or an ELK query and clear instructions on how to interpret the results to determine whether the problem is still occurring. Check the "Verification" section in this issue as an example of this.
 1. Alternatively, provide clear instructions on how to recreate or validate the problem.
 1. If an issue has been resolved, use the following process:
 1. Reassign the issue back to the author, or an appropriate owner, requesting that they confirm the resolution, and close the issue if they concur. If not, they should follow up with a note and unassign themselves.

SECTION: engineering/workflow/code-review.md

Overview

Code reviews are mandatory for every merge request, you should get familiar with and follow our Code Review Guidelines.

These guidelines also describe who would need to review, approve and merge your, or a community member's, merge request. They also describe the review response time SLO's team members have to abide by.

Values

Every reviewer at GitLab must strive for our reviewer values.

Reviewer

All GitLab engineers can, and are encouraged to, perform a code review on merge requests of colleagues and community contributors. If you want to review merge requests, you can wait until someone assigns you one, but you are also more than welcome to browse the list of open merge requests and leave any feedback or questions you may have.

You can find someone to review your merge requests by looking on the team page, or on the list of GitLab Engineering Projects, both of which are fed by YAML files under `data/teammembers/person/`.

You can also help community contributors get their merge requests ready, by becoming a Merge

Request Coach.

Note that while all engineers can review all merge requests, the ability to accept merge requests is restricted to maintainers.

Maintainer

Maintainers are GitLab engineers who are experts at code review, know the GitLab product and codebase very well, and are empowered to accept merge requests in one or several GitLab Engineering Projects.

Every project should have at least two maintainers, but most should have more. Some projects have separate maintainers for different specialties. For example, the main GitLab codebase has separate maintainers for frontend, backend, and database.

Great engineers are often also great reviewers, but code review is a skill in and of itself and not every engineer, no matter their seniority, will have had the same opportunities to hone that skill. It's also important to note that a big part of being a good maintainer comes from knowing the existing product and codebase extremely well, which lets them spot inconsistencies, edge cases, or non-obvious interactions with other features that would otherwise be missed easily.

Becoming a reviewer/maintainer means you are taking on a broader responsibility beyond your immediate group. Your available capacity should be adjusted accordingly to make room for you to work effectively. There is no strict formula for this as each project comes with a different workload, but do make sure to discuss this with your manager to avoid burnout and to ensure your manager understands how this may impact your team's capacity.

To protect and ensure the quality of the codebase and the product as a whole, people become maintainers only once they have convincingly demonstrated that their reviewing skills are at a comparable level to those of existing maintainers.

As with regular reviewers, maintainers can be found on the team page, or on the list of GitLab Engineering Projects.

Senior+ Maintainers

For senior+ engineers, being a maintainer is part of their job family unless they have discussed with their manager or Team Member Relations in line with our reasonable accommodation process. Any of the engineering projects considered as part of the product are suitable, provided the individual has been a contributor and an active reviewer of the project. Effective 2022-08-01, we use the following table for the timeframe expectations of maintainership:

Description	Timeframe
-------------	-----------

-----	-----
-------	-------

Intermediate Engineers	Maintainership is optional
------------------------	----------------------------

Existing Senior+ Engineer	Existing senior+ engineers who are not already maintainers are encouraged to complete the trainee program in support of our team's productivity and motivation. There is no expected completion timeframe of the trainee program.
---------------------------	---

Newly hired Senior+	During onboarding, newly hired senior+ engineers will be asked to
---------------------	---

become trainee maintainers instead of reviewers. We expect their maintainership to be complete within 12 months of their onboarding completion. |

| Promotions to Senior | For engineers moving into the Senior role, we expect that they have already become a maintainer prior to promotion. |

Meeting the reviewer/maintainer

Communication happens easier when you are familiar with the person reviewing the code. Take opportunities (for example coffee chats) to get to know reviewers to break the ice and facilitate future communication.

How to become a project maintainer

This applies specifically to backend, frontend and database maintainers. Other areas (docs, etc.) may have separate processes documented below.

Before considering maintainership, first you should be a contributor. You should have made at least a few feature or maintenance contributions to the project before you can become a reviewer in the trainee maintainer process. These contributions should be complex enough to give you an understanding of the project's unique domain and design.

Maintainership check-ins and mentorship

Interested reviewers should check in regularly with their manager/mentor to discuss progress towards maintainership and review any recent detailed reviews, for example during their 1-on-1s. Reviewers are encouraged to also seek out a maintainer mentor for further perspective on their reviews. Reviewers are encouraged to think of their eligibility for maintainership in the terms of "I could be ready at any time to be a maintainer as long as it is justified".

You can also open a trainee maintainer issue using this template allowing you to build up examples that will translate into your final merge request.

Merge request feedback for reviewers

After each review is complete, the reviewer should write up a justification about why they believe the merge request is ready to merge. This justification is then reviewed by the maintainer and if the maintainer agrees with the justification they should add a 🟢 reaction to the comment, even if they have additional non-blocking comments. The maintainer should leave a comment highlighting any blocking concerns that were missed in the initial review.

Maintainership nomination process

At any time, the manager/mentor may choose to open a merge request, adding the reviewer as a maintainer. This merge request should have a justification from the manager/mentor as to why the reviewer should become a maintainer. You are also welcome to open this merge request yourself at any time. There are merge request templates available to help you with the content and steps.

Pre-requisites for Maintainership

Before opening the merge request, the author should:

1. Review justifications for several of the reviewer's recent merge requests.
1. Reach out to at least two of the maintainers privately for feedback on the reviewer. The reviewer may have some suggestions on who these maintainers could be.

Request Maintainership feedback

Before merging, the manager/mentor should:

1. Mention the maintainers from the given specialty and ask them to provide feedback to the manager/mentor directly. Emphasize that any negative feedback should be communicated privately to the manager/mentor, not in the merge request. This is inline with our Collaboration value that Negative feedback is 1-1. Refer to the additional guidance for managing negative feedback.
1. Leave the merge request open for 1 week, to give the maintainers time to provide feedback to the manager/mentor.
1. Have at least 2 approvals from existing maintainers.

Managing negative feedback

If the manager/mentor receives private feedback indicating the reviewer is not ready to become a maintainer:

1. The manager/mentor should review the concerns raised and decide whether it's substantial enough to close the merge request.
1. The manager/mentor could then close the merge request with a comment about there being feedback for the reviewer to work on, but keep the feedback confidential.
1. The manager/mentor would provide the feedback directly to the reviewer in a one-to-one conversation. This approach allows the reviewer to address the gaps before being re-submitted for maintainer status. The earlier the manager/mentor can solicit and receive this feedback, the better.

Handling disagreements in maintainer readiness:

The manager/mentor should seek to understand any concern raised by a current maintainer when they disagree with a trainee maintainer's readiness or qualifications. Use the following guidelines to determine if a single disapproval should veto approvals received in favor of the trainee maintainer.

1. In keeping with other values, the maintainer's concern(s) should not be personal or prejudicial.
1. The maintainer's concern(s) must be consistent with the Responsibilities of a Maintainer.
1. The maintainer's concern(s) should be grounded in fact that:
 1. the trainee maintainer consistently does not perform MR reviews in a conventional manner, or
 1. the trainee maintainer has consistently been irresponsible in ensuring our code quality and standards as isolated incidents are expected in the training process.

In order to better inform a decision, the manager should privately reach out to 2 existing maintainers without sharing any personal information regarding the feedback. The manager is ultimately responsible for the readiness of the trainee maintainer and owns the decision to

entrust the trainee maintainer with maintainer responsibilities.

Maintainership approval

After merging, the manager should:

1. Announce this change in the applicable channels listed under Slack section of the engineering communications handbook and backendmaintainers/frontendmaintainers and backend/frontend.
1. Post an update in the Engineering Week-in-Review document. The agenda is internal only, please search in Google Drive for 'Engineering Week-in-Review'.

Additional steps for approved project maintainers

Interested reviewers for the projects below should complete the listed tasks in addition to what is described in How to become a project maintainer to progress from a reviewer to a maintainer.

Project maintainer process for gitlab-rails

- For Backend Maintainers, ping to @gitlab-org/maintainers/rails-backend
- For Frontend Maintainers, ping to @gitlab-org/maintainers/rails-frontend

Project maintainer process for gitlab-database

- Familiarize with database review process.
- Familiarize with migration helpers and review usage in existing migrations.
- Familiarize with best practices in database guides.
- Read Understanding EXPLAIN plans.
- Get yourself added to @gl-database group and respond to @-mentions to the group (reach out to any maintainer on the group to get added). You will get TODOs on gitlab.com for group mentions..
- Create an access request for psql/AllFeaturesUser access to database lab/postgres.ai if you do not already have AllFeaturesUser access.

Tips:

- If you'd like a dashboard limited to just reviews where you applied the ~"database::reviewed" label, reach out to the database group manager to get one.

Project maintainer process for gitlab-components

gitlab-components

- Review our documentation and best practices for creating CI/CD components.
- Familiarize with our documentation on GitLab-maintained components.
- Develop one or more components to familiarize yourself with the development process.
- Create a merge request and assign the merge request to your manager for you to be added as a CI/CD components maintainer. Be sure to cite references for the CI/CD components you have

developed. (See MR example)

- Ping the existing maintainers (@gitlab-org/maintainers/ci-components) in the MR for review and feedback.
- Have at least 2 approvals from existing maintainers in order to merge the MR.

Upon approval, the maintainer who merges the MR will:

- Add the newly approved maintainer to the CI components maintainer group (@gitlab-org/maintainers/ci-components).
- Announce in cicomponentsmaintainers and post an update in the Engineering Week-in-Review document

Project maintainer process for design.gitlab.com or gitlab-svgs

design.gitlab.com or gitlab-svgs

- Understand how to become a maintainer.
- All designers are maintainers of gitlab-design project. If you are interested in becoming a Maintainer of UI (.scss) for gitlab, and gitlab-ui projects, please follow the Engineering Review Workflow.
- It is up to you to ensure that you are getting enough MRs to review, and of varied types. You could also seek out more reviews from your team, for example by asking for reviews on the ux and pajamas-design-system Slack channels. If you are not receiving enough MRs to advance in your training, be proactive and work on your own improvements to Pajamas. This will demonstrate overall understanding of the product, as well as quality contributions, and help propel your progress. Maintainers are available to help guide you.
- Your reviews should aim to cover maintainer responsibilities as well as reviewer responsibilities. Design Maintainers should be focused on MRs that have an impact on usability, iterate on existing user experience, and/or include usage of design guidelines, standards, and patterns. Your approval means you think it is ready to merge.
- As maintainer, you must keep relying on others for the expertise they have that you don't. In the MR description, highlight the efforts that showcase your results at a maintainer-level, what skills you want to continue working on, and add a link to this issue. (Example).

Project maintainer process for gitlab-quality

gitlab-quality

- Choose which Quality project(s) you would like to become a maintainer for:
 - GitLab (/qa)
 - GitLab-QA
 - GitLab Triage
 - Triage Ops
 - GitLab CustomersDot (/qa)
 - GitLab Environment Toolkit (GET)
 - GitLab Performance Tool (GPT)
- The requirements for each project is different, please make sure to read the maintainer section in the Quality handbook.

Project maintainer process for gitlab-secure-analyzers

gitlab-secure-analyzers

- Understand our Secure Team standards and style guidelines.
- Understand our Secure Release Process.
- Understand our Secure QA Process.

Project maintainer process for gitlab-elasticsearch-indexer

gitlab-elasticsearch-indexer

- Complete go lang training.
- Review the GitLab Elasticsearch Indexer development and release process.
- Join the gglobalsearch Slack channel.
- Work on issues to gain familiarity with the project.
- Optional: Reach out to an existing maintainer to help you become a maintainer.

Project maintainer process for customers-gitlab-com

customers-gitlab-com

- Understand our standards and style guidelines.
- Understand the software architecture used in Fulfillment systems.
- Read through CustomersDot documentation.
- Contribute to issues and gain familiarity with the project.
- Contribute to reviews demonstrating domain expertise and consistency with reviewer responsibilities.

Project maintainer process for gitlab-secure-license-db

gitlab-secure-license-db

- Familiarize with GitLab Go standards and style guidelines.
 - Complete Golang training issue if you don't have prior experience with Go.
 - Watch External License DB Architecture and Repository Walkthrough
 - Review Fullstack Development of LicenseDB guideline.
 - Understand how to release and deploy new changes to the components.
 - Understand how the scheduled pipelines are used for the deployment project.
 - Author or review 3 merge requests in total to a specific project in the license-db namespace.
- Maintainership is granted per project.

Project maintainer process for gitlab-chart

gitlab-chart

- Familiarize yourself with Distribution's merge request workflow
- Familiarize yourself with the architecture and style guide of the GitLab Helm chart.
- Understand the relation between the GitLab Operator and the GitLab Helm chart.
- Contribute to issues and review merge requests.
- Understand how the GitLab Helm chart is tested using rspec.

Project maintainer process for gitlab-operator

gitlab-operator

- Familiarize yourself with Distribution's merge request workflow
- Familiarize yourself with GitLab Go standards and style guidelines.
- Understand how custom resources and controllers work.
- Familiarize yourself with the following libraries and tools:
 - operator-sdk
 - controller-runtime
 - Operator Lifecycle Manager
 - envtest
- Contribute to issues and review merge requests.
- Understand the relation between the GitLab Operator and the GitLab Helm chart.

Project maintainer process for ai-gateway

ai-gateway

- Understand the software architecture.
- Setup GitLab Duo for local development.
- Read through Maintainership documentation and follow the steps outlined there to become a maintainer.

Learning to be a maintainer

While any reviewer may be recommended by their manager to become a maintainer at any time, reviewers who wish to become maintainers should follow a few basic steps on each review in order to get into a maintainer mindset, and learn from feedback from maintainers.

Create a merge request and indicate your role as a project-name: traineemaintainer in your team member entry. Assign MR to your manager for merge.

After each review, reviewers should summarize why they believe a merge request is ready to be merged:

For example:

> Looks good! I believe this MR resolves the issue and it looks safe because the code change is relatively isolated.
>
> LGTM! I feel this MR is a good iteration. And it has low risk because it is behind a feature flag.

Maintainers should respond to the comment from the reviewer with a · if they agree, and upon merging if there were additional comments they feel should have been caught, they should ping any reviewers so they are aware of the comments.

Some reviewers find it helpful to track their progress. This is not required, but a few ways people have done this are:

- Keep an issue with all of the various reviews and feedback comments from maintainers they have received. There are some tools to help with this type of issue:
 - <https://gitlab.com/nolith/review-tanuki>
 - <https://gitlab.com/caalberts/review-tanuki>
 - <https://gitlab.com/arturoherrero/trainee>
 - <https://gitlab.com/knejad/gitlab-career-developmentmr-reviews>
- Use an emoji to mark all MRs they received feedback from maintainers on so they are easily searchable.

After becoming a maintainer

If you've become a new maintainer, follow these instructions to request relevant permissions that will allow you to fulfill your role:

- Join the maintainer's group channel on Slack: frontendmaintainers, backendmaintainers, etc.
- Ask the maintainers in your group to invite you to any maintainer-specific meeting if one exists.
- Request access to the GitLab maintainer group you belong: frontend, backend, or database.
- Request maintainer permissions on the projects you will act as a maintainer using the Single Person Access Request issue template. Once you've created the issue, request another maintainer to grant you those permissions.

Reviewer mentorship program

Training and onboarding new maintainers is an important process. As the engineering team grows and the total number of MRs rapidly expands, the number of MR reviews per maintainer quickly becomes unsustainable.

Recent research has shown that the trainee process for new maintainers is hindered by several key factors:

- Reviewers self-perceived readiness and confidence
- Reviewers ability to review a substantial quantity of MRs
- Reviewers ability to review a diversity of MRs covering enough breadth across the codebase

Structure

1. Participation is voluntary for both maintainers and reviewers.
1. Maintainers may directly mentor up to 4 reviewers at a time.
1. Mentor / reviewer assignments are coordinated within a maintainermentorship.yml file.
1. New reviewers need to locate an existing maintainer that has an opening available in maintainermentorship.yml and contact the maintainer directly.
1. Every 6 weeks the maintainer will check-in with each reviewer. This could happen async or via Coffee chat.
1. The goal of the checkin is to: review MRs, answer questions, clarify any doubts, and track readiness toward graduating.
1. Mentorship is capped at 12 months by which the reviewer should be prepared to graduate.
1. At least 1 additional check-in should be scheduled once the reviewer has graduated to celebrate the achievement and answer any further questions.
1. Make sure to remove graduated reviewers from the maintainermentorship.yml file to make space

for new mentees.

Benefits

1. It develops and expands mentorship skills among maintainers
1. Gives reviewers a regular touch point for skilling-up in deficient areas
1. Creates stronger networks amongst reviewers/maintainers than currently exists
1. Mentoring directly under a maintainer should catalyze more competence and confidence in taking ownership of the GitLab codebase.

Transitioning away from being a reviewer/maintainer

After consultation with your manager, you may wish or need to transition away from being a reviewer/maintainer. Regardless of the circumstances, it's perfectly OK for this to happen! Responsibilities and workloads change; projects evolve. So it's important to ensure your time is spent on the areas that are most important. To make the change official and to be removed from reviewer roulette:

1. See the Team Member Database document for how to update your YAML file.
1. Create a new MR and assign the MR to your manager.

Returning to become a reviewer/maintainer

After a period of transitioning away from being a reviewer/maintainer, you may wish to return to performing these duties. To make the request official, see the section on how to become a project maintainer, creating the tracking issue and Merge Request, referencing previous tracking issue(s) and Merge Request(s) for context. The newly created Merge Request should be assigned to your manager for immediate review as the process can usually be fast tracked.

Maintainership process for smaller projects

This is a general purpose template that projects belonging to a specific group and/or projects with less than 10 internal contributors can use to define and document their maintainership process.

Projects may adopt these guidelines for maintainership to help grow maintainers in projects where there are not enough maintainers.

- All team members should consult the engineering development roles and become a reviewer or maintainer.
- Add a project specific maintainer process under How to become a project maintainer. A lightweight template is provided.
- Enable Danger Review using simplerolette within the project to identify MR reviewers.
- Reduce the number of Merge Request reviews required to be considered a maintainer.
- Count work on the project itself as progress for maintainership.
- Require a maintainer mentor to help speed up the process.
- Curate some practice MRs based on past reviews. Create copies of the MRs in a closed state

and provide the links in the project maintainership process.

- Consider creating project-specific development guidelines if they don't already exist for the project.
- Add instructions on how to become a project maintainer to the project README.

Accelerated maintainer onboarding for smaller projects

Smaller projects with a limited number of maintainers can benefit from an accelerated onboarding process. This process is less involved than the main GitLab maintainership, mainly because there is not enough feature work and MRs in these smaller projects. Each project can modify this process to fit their needs.

The onboarding process consists of the following steps:

- Establish the criteria for becoming a maintainer and the process for onboarding new maintainers and document them on an issue. For example:
 - Eligible engineers: Everyone who wants to work with TypeScript.
 - Training: Recorded code walk-through and interactive pairing sessions.
 - Criteria for becoming a maintainer: Self-selection after the training based on confidence in the codebase.
- Share knowledge by pair programming on an end-to-end feature. A current project maintainer will host a series of pairing sessions to educate potential maintainers. In the VS Code Extension pilot, we changed the implementation of one of the VS Code commands in four one-hour sessions spread over four weeks.
- Schedule the code walk-through and pairing sessions with potential new maintainers. Determine the appropriate amount of time and number of sessions needed to provide a comprehensive understanding of the codebase.
- In the pilot program, we also considered that each potential maintainer will do X MR reviews, but there wasn't enough MRs in the project.
- After you design the process, ask for feedback from potential new maintainers. They might prefer a different mix of walk-throughs, pairing sessions and other resources.

By following this onboarding process, projects can efficiently add new maintainers with a solid understanding of the codebase and the project's business logic. An example of this process in action can be seen in the GitLab VS Code Extension Project.

Maintainer process template

Use this lightweight template as a starting point for defining your project's maintainer requirements.

- Work on issues to gain familiarity with the project's domain and guidelines.
- Complete project-specific language training if the programming language is not used in the main GitLab project (example: go lang training).
- Review project-specific release process (if one exists).
- Join the [project or team] Slack channel.
- Create a merge request and indicate your role as a project-name: traineemaintainer in your team member entry. Assign MR to your manager for merge.
- Optional: Perform simulated reviews on practice MRs if available.
- Optional: Peer review MRs.

- Optional: Reach out to an existing maintainer to help you become a maintainer.

Maintainer ratios

We aim to keep the engineer : maintainer ratio under 6, for both frontend and backend. We track this in the Engineer : Maintainer Ratio dashboard:

<https://10az.online.tableau.com/t/gitlab/views/DraftEngineerMaintainerRatio/EngineerMaintainerRatio1>

Maintainer Demand

We can gauge demand by looking at the Maintainership Demand dashboard, which can be filtered by month, project and technology:

<https://10az.online.tableau.com//site/gitlab/views/MaintainershipDemand/MaintainershipDemand?id=1>

About this dashboard

Definitions of metrics:

- Incoming Merge Requests - The total number of Merge Requests opened by any author.
- Average Availability - The average percentage of time that maintainers are accepting reviews, and not busy, out of office, on vacation, or at capacity.
- Total Maintainers - The number of engineers who have maintainer specified in their team.yml
- Available Maintainers - The number of Total Maintainers multiplied by the Average Availability
- Monthly Review Target - A variable number based on the expected number of reviews per maintainer in a given month. Monthly review targets can be custom based on the area, or generally defined based on the number of Incoming Merge Requests. For more information on this metric, read below.
- Target Maintainer Counts - A variable number, changing month over month, based on the number of Incoming Merge Requests to the area for that month, the current number of Total Maintainers, and the Average Availability.
- Minimum Maintainers Required - The number of Available Maintainers required in order to meet the demand of Incoming Merge Requests.
- Maintainers Needed - The number of Available Maintainers still needed in order to meet Minimum Maintainers Required.
- Technology Group - For projects that have specific specialties for their maintainers, like database, backend, frontend, CI Templates, Workhorse

Explanation of charts:

- Available Maintainers VS Need - An overall, high-level look at our maintainer targets, totals, and availability.
- Projects where maintainers are needed - Based on the previous months data, this is an output of any area predicted to need more available maintainers.
- Project/Area Maintainership Health - The percentage of projects (GitLab) or areas (backend) that are not meeting the Minimum Maintainers Required over time.

- Unhealthy Core Areas of Maintainership Health - The percentage of projects (GitLab) or areas (backend) that are not meeting the Minimum Maintainers Required, and are receiving more than 100 Incoming Merge Requests per month over time.
- Part of Product Repos - Full Data - An all-inclusive chart for each project, area, and month that includes all metrics noted above.

Monthly Review Targets

Targets are calculated based on the number of available maintainers (described above) and what a "reasonable" number of reviews per maintainer per month is. "Reasonable" has been defined for some areas in separate analysis issues. These are custom targets defined in the "maintainercustomtargets" Sisense snippet. There are general targets for all other projects based on the number of incoming merge requests to the project. These numbers are a first iteration and were based on the analysis issues, where less demanding projects had fewer maintainers (therefore requiring more monthly reviews per person) and more demanding projects had more maintainers (therefore requiring less monthly reviews per person):

- The default target is 5 reviews per maintainer
- If the area receives more than 10 merge requests, the monthly target is 10 reviews per maintainer
- If the area receives more than 500 merge requests, the monthly target is 40 reviews per maintainer
- If the area receives more than 1000 merge requests, the monthly target is 16 reviews per maintainer
- If the area receives more than 1500 merge requests, the monthly target is 20 reviews per maintainer

To add a custom target to an area using the maintainercustomtargets Sisense snippet:

- In Sisense, go to Snippets > maintainercustomtargets
- Add a new CASE/WHEN statement based on the project name and optionally technology group
- Set the number to the ideal monthly review target per person, according to what a reasonable number of reviews is for this project

Caveats

- Total Maintainer counts may show as 0 - This data is using reviewer roulette to determine the number of total maintainers because we do not have access to the project memberships in Sisense, and because many projects have maintainers/owners who are not truly active. One reason this may be showing 0 despite having maintainers on the project is because the project name displayed does not match the project name used in the team.yml for reviewer roulette. Another reason might be that the project is not using reviewer roulette. In these cases, the project will need to be set up and configured correctly to use reviewer roulette. Finally, the team.yml must match the requirements of the project or area - for example, if reviews in your project are able to go to any maintainer, the team.yml should specify maintainer. If the reviews in your project are separated by specialty, the team.yml must specify maintainer [SPECIALTY] and the merge request should be labeled according to that specialty.

Maintainer/Reviewer Availability

We aim to have enough reviewers and maintainers across timezones to ensure that there are people available to review MRs in a timely manner while keeping review load at a reasonable level. We track this in the Reviewer/Maintainer Availability and Capacity dashboard:

<https://10az.online.tableau.com//site/gitlab/workbooks/2286852/views>

Leading Organizations

All wider community members, and their organizations, can contribute. We strongly believe that contributing to open source, and particularly to GitLab is a competitive advantage for organizations and their members, and want to reward those organizations for doing so. GitLab highly encourages, celebrates & rewards those that contribute in frequent and atomic iterations. When an organization or individual without affiliations reaches 20 merged merge requests or more within the last 3 completed months, we consider that organization or individual a Leading Organization.

Organizations are matched based on the Organization field in your profile. GitLab can also match individuals to organizations using other metadata available to the Contributor Success team. Create an issue in the Contributor Success queue if you think you or your organization should qualify but is not receiving the label Leading Organization on your merge requests.

A Leading Organization is entitled to a review response SLO. This entitlement will be awarded in the beginning of each month. Merge requests that were created during the time in which an organization is entitled to the Leading Organization status, will receive the label Leading Organization.

> Leading Organization = 20 merged Merge Requests or more within the last 3 completed months.

Eligible merge requests include contributions to the GitLab product and documentation. Contributions to the www.gitlab.com repository (e.g. the GitLab handbook) are not currently included or entitled to a review response SLO.

Domain Experts

Our Code Review Guidelines states that we default to assigning reviews to team members with domain expertise.

What makes a domain expert?

We currently don't provide rigid rules for what qualifies a team member as a domain expert and instead we use a boring solution of implicit and self-identification.

Implicit:

- Team members working in a specific stage/group (e.g. Create: Source Code) are implicitly

considered domain experts for that area of the app they work on.

- Team members working on a specific feature (e.g. search) are implicitly considered domain experts for that feature.

Self-identification:

- Team members can self-identify as a domain expert for a specific feature (e.g. file uploads).
- Team members can self-identify as a domain expert for a specific technology (e.g. GraphQL), product feature (e.g. file uploads) or area of the codebase (e.g. CI).

How to self-identify as a domain expert

The only requirement to be considered a domain expert is to have substantial experience with a specific technology, product feature or area of the codebase. We leave it up to the team member to decide whether they meet this criteria.

1. Define a new, or use an existing domain expertise key, located in data/domainexpertise.yml.
1. Update your entry in your own YAML file with a new domainexpertise property and list all domain expertise keys.

Example:

domainexpertise.yml

```
yaml
webpack:
  displayname: Webpack
  link: https://webpack.js.org/
frontendarchitecture:
  displayname: Frontend Architecture
  link: https://docs.gitlab.com/ee/development/feguide/architecture.html
```

yourhandle.yml

```
yaml
domainexpertise:
  - webpack
  - frontendarchitecture
```

When self-identifying as a domain expert, it is recommended to assign the MR to be merged by an already established Domain Expert or a corresponding Engineering Manager.

Where can I find a list of people with domain expertise?

The expertise of a team member can be seen on the Engineering Projects page.

Review turnaround time

Because unblocking others is always a top priority, reviewers are expected to review merge requests in a timely manner, even when this may negatively impact their other tasks and priorities.

Doing so allows everyone involved in the merge request to iterate faster as the context is fresh in memory, and improves contributors' experience significantly.

Review-response SLO

To ensure swift feedback to ready-to-review code, we maintain a Review-response Service-level Objective (SLO).

The SLO applies to GitLab team members and Leading Organizations, but not to other wider community contributors.

The SLO is defined as:

> Review-response SLO = (time when review is provided) - (time MR is assigned to reviewer)

The SLO value depends on the author of the merge request:

- From GitLab team members: Review-response SLO < 2 business days
- From authors of Leading Organizations: Review-response SLO < 4 business days

If you don't think you can review a merge request in the Review-response SLO time frame, let the author know as soon as possible in the comments (no later than 36 hours after first receiving the review request) and try to help them find another reviewer or maintainer who is able to, so that they can be unblocked and get on with their work quickly. Remove yourself as a reviewer.

Reviewers may also communicate their status through the use of several other emoji. For more details on these other statuses, please refer to the code review page in the developer documentation.

Of course, if you are out of office and have communicated this through your GitLab.com Status, authors are expected to realize this and find a different reviewer themselves.

When a merge request author has been blocked for longer than the Review-response SLO, they are free to remind the reviewer through Slack or add another reviewer.

Managing expectation

When you are assigned to review an MR and you are not able to get to it within the Review-response SLO, you should leave a comment on the MR informing the author of your delayed response. If possible, you should also indicate when the author can expect your feedback or help them find an alternative reviewer.

As the author of an MR you should reassign to another reviewer or maintainer if the Review-response SLO has not been met and you have been unable to contact the assignee.

Code Owner approvals

Some GitLab projects use GitLab's CODEOWNERS file feature to manage approvals for specific file paths and types. In the gitlab-org/gitlab project, we use a combination of CODEOWNERS approval rules plus MR approval settings in order to follow segregation of duties best practices. This section describes the process for updating the eligible approvers for CODEOWNERS changes for the gitlab-org/gitlab project.

The Code Owners for the CODEOWNERS file itself are managed with a rule in the file. For example:

text

```
CODEOWNERS @gitlab-org/development-leaders @gitlab-org/tw-leadership
```

There are two ways to update the Code Owner(s) of the CODEOWNERS file:

1. Update the membership of a group that already has the ability to approve CODEOWNERS changes via the standard access request process.

1. Open a merge request to update the relevant lines. An existing Code Owner will have to approve the merge request. You are also encouraged to ping a security compliance team member for visibility.

The @gitlab-org/development-leaders group consists of team members from Senior Managers and above in the management track, and Distinguished Engineer and above in the individual contributor track in the development departments within Engineering.

SECTION: engineering/workflow/iteration.md

In order to provide changes in an iterative and incremental manner, we should always seek to create smaller merge requests.

Engineers are a vital part of a feedback cycle with product management to design an MVC as engineers understand the technical challenges that additional behaviours/edge cases have on the effort to ship something. You should always attempt to provide feedback as early as possible to product management if you can see opportunity to cut scope and deliver a smaller change no matter where you are at in the planning/development lifecycle.

How to keep a merge request small

- Consider extracting edge-cases that aren't an impediment to introducing the smallest working piece of functionality into follow-up issues. This will save the merge request from growing during a review. The follow-up issues are also useful for distributing the work among the team when the issue turned out to be more complex than expected.

- Refactorings that require a complex approach can be introduced separately either before the actual changes or as a follow-up in case the quality doesn't suffer from the introduced changes.

- If your issue/merge request description or comment says "First we should do this, make sure that works, refine this", it is also a sign of a possible split that happens before work even begins. In general, if an engineer is following the "one commit, one logical change" model, then each commit can potentially be a separate merge request.

Tradeoffs between horizontal and vertical slicing

Developers sometimes think of Horizontal slicing as the only option to create small merge requests. They separate their changes along horizontal, tech-stack-based lines: splitting out database changes from backend changes from frontend changes (and so on). However, this approach doesn't always mean it will be more efficient and can make it difficult for reviewers to provide meaningful feedback when reviewing the separate layers if they cannot see the complete picture and as such adds additional risk of merging code without easily seeing the full context. Before starting the work on an merge request that involves multiple layers of the stack, consider having a conversation with your team around which approach will work best.

Horizontal slicing

1. Works well when each part is going to be large, complex change and there is no clear way to break down the MVC further.

1. Works well for merge requests where each layer of the stack will be isolated and requires little to no context from the reviewers looking at the layers of the stack independently.

1. When doing this you should ensure the overall change is planned and explained to the reviewer somewhere linked to from the merge request or clearly detailed in the Merge Request description.

1. Avoid if possible when each layer of the stack is dependent on others to fully understand the change. For example, since changes to ~database (ie. database migrations written) are tightly coupled to the related ~backend changes (the code which queries the new database columns/tables etc.), it is still preferable to ensure they are part of the same MR as both the ~database reviewer and ~backend reviewer will need to review both of those code changes in order to provide meaningful feedback.

Vertical slicing

1. When doing this you should ensure the overall merge request size will remain within acceptable review range.

1. Works well when each layer of the stack is fairly dependent on another and seeing them together will help reviewers.

1. Works well when each part is succinct and rely on one another.

1. Avoid if each layer of the stack will require complex change that spans over multiple merge requests as it can get hard to follow. If this is the case, you should still try to break down the MVC fu